

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №7
по курсу «Алгоритмы и структуры данных»
Тема: Динамическое программирование №1
Вариант 7 (Продвинутый уровень)

Выполнила:
Кулакова Елизавета Алексеевна
К3260

Проверил:
Харлуниин А. А.

Санкт-Петербург
2025 г.

Содержание отчета

Содержание отчета.....	2
Задачи по варианту (базовый уровень).....	3
Задача №1. Обмен монет.....	3
Задача №7. Обработка сетевых пакетов.....	8
Задачи на продвинутый уровень.....	14
Задача №3. Редакционное расстояние.....	14
Задача №4. Наибольшая подпоследовательность двух последовательностей.....	22
Вывод.....	27

Задачи по варианту (базовый уровень)

Задача №1. Обмен монет

Как мы уже поняли из лекции, не всегда "жадное" решение задачи на обмен монет работает корректно для разных наборов номиналов монет. Например, если доступны номиналы 1, 3 и 4, жадный алгоритм поменяет 6 центов, используя три монеты (4 + 1 + 1), в то время как его можно изменить, используя всего две монеты (3 + 3). Теперь ваша цель - применить динамическое программирование для решения задачи про обмен монет для разных номиналов.

- **Формат ввода / входного файла (input.txt).** Целое число *money* ($1 \leq money \leq 10^3$). Набор монет: количество возможных монет *k* и сам набор $coins = \{coin_1, \dots, coin_k\}$. $1 \leq k \leq 100$, $1 \leq coin_i \leq 10^3$. Проверку можно сделать на наборе {1, 3, 4}. Формат ввода: первая строка содержит через пробел *money* и *k*; вторая - $coin_1 coin_2 \dots coin_k$.

– **Вариация 2:** Количество монет в кассе ограничено. Для каждой монеты из набора $coins = \{coin_1, \dots, coin_k\}$ есть соответствующее целое число - количество монет в кассе данного номинала $c = \{c_1, \dots, c_k\}$. Если они закончились, то выдать данную монету невозможно.

- **Формат вывода / выходного файла (output.txt).** Вывести одно число – минимальное количество необходимых монет для размена *money* доступным набором монет *coins*.
- Ограничение по времени. 1 сек.
- Примеры:

input.txt	output.txt	input.txt	output.txt
2 3	2	34 3	9
1 3 4		1 3 4	

Листинг кода

```
from generator import *
from measurments import *

@measure_performance
def dpchange(money, coins):
    mnc = [float('inf') for _ in range(0, money+1)]
    mnc[0] = 0
    for m in range(1, money+1):
        for i in range(len(coins)):
            if m >= coins[i]:
                mnc[m] = min(mnc[m], mnc[m - coins[i]] + 1)
    return mnc[money] if mnc[money] != float('inf') else -1

@measure_performance
def limited_dpchange(money, coins, lims):
    dp = [float('inf') for _ in range(0, money + 1)]
    dp[0] = 0
    for coin, cnt in zip(coins, lims):
        k = 1
        while cnt > 0:
            take = min(k, cnt)
            w = take * coin
```

```

        amount = take
        for x in range(money, w - 1, -1):
            dp[x] = min(dp[x], dp[x - w] + amount)
        cnt -= take
        k <= 1

    if dp[money] == float('inf'):
        return -1
    return dp[money]

if __name__ == '__main__':
    cash_generator(10**3, 100)
    with open('input.txt') as f:
        s, _ = map(int, f.readline().split())
        cs = list(map(int, f.readline().split()))
        lmts = list(map(int, f.readline().split()))
    res = dpchange(s, cs)
    res_limited = limited_dpchange(s, cs, lmts)
    with open('output.txt', 'w') as f:
        f.write(str(res))
    print(res, res_limited)

```

Листинг кода функции для формирования массива

```

def cash_generator(s=None, n=None):
    if s is None:
        s = random.randint(1, 1000)
    if n is None:
        n = random.randint(1, 100)

    limits = random.choices(list(range(1, 101)), k=n)
    cash = sorted(np.random.randint(1, 1001, n))
    with open('input.txt', 'w') as f:
        f.write(f'{s} {n}\n')
        f.write(f'{' '.join(map(str, cash))}\n')
        f.write(f'{" ".join(map(str, limits))}')

```

Текстовое объяснение решения.

Для неограниченной версии задачи используется одномерный массив `mnc`, где элемент `mnc[x]` хранит минимальное количество монет, необходимое для получения суммы `x`. Базовое состояние `mnc[0] = 0` соответствует нулевой сумме без монет, а все остальные значения инициализируются бесконечностью как признак недостижимости. Переход строится по принципу «последнего шага»: если последняя взятая монета имеет номинал `coin`, то до её добавления должна была быть достигнута сумма `x - coin`. Именно поэтому используется индекс `mnc[x - coin]`, а не произвольное значение. Перебор суммы ведётся по возрастанию, что корректно для неограниченной версии, поскольку один и тот же номинал может использоваться сколько угодно раз. Если после завершения

вычислений значение `mnc[money]` остаётся бесконечным, размен невозможен и возвращается -1.

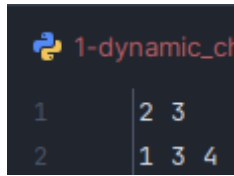
Во второй вариации задачи количество монет каждого номинала ограничено, поэтому прямое применение предыдущего алгоритма становится некорректным: при прямом проходе динамика позволила бы использовать одну и ту же монету несколько раз. Для решения этой проблемы применяется бинарное разложение количества монет. Каждое ограничение `snt` представляется в виде суммы степеней двойки (1, 2, 4, ...), что позволяет заменить один номинал с ограничением на набор «блоков», каждый из которых может быть использован не более одного раза. Каждый такой блок характеризуется вкладом в сумму $w = \text{take} * \text{coin}$ и стоимостью `take` в терминах количества монет. После этого задача сводится к 0/1-рюкзак с минимизацией числа предметов.

Ключевым моментом реализации ограниченной версии является обратный проход по сумме `x` от `money` к `w`. Именно этот «манёвр» предотвращает повторное использование одного и того же блока в пределах одной итерации: при обратном проходе состояние `dp[x - w]` гарантированно относится к результатам без текущего блока. Индекс `x - w` появляется из той же логики «последнего шага»: если последний шаг добавляет вклад `w`, то предыдущее достижимое состояние обязано соответствовать сумме `x - w`. Таким образом, корректно соблюдается ограничение на количество монет каждого номинала.

Отдельно стоит отметить поведение неограниченной версии при случайно сгенерированных входных данных. При большом числе номиналов и случайном распределении значений почти всегда находится разложение суммы в виде суммы двух номиналов, поэтому результат часто равен 2. Это не является ошибкой алгоритма, а объясняется вероятностными свойствами плотного случайного множества номиналов и тем фактом, что задача минимизирует именно количество монет.

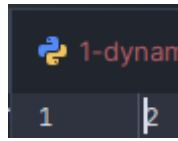
Экспериментальные замеры времени и памяти показывают, что оба алгоритма уверенно укладываются в заданные ограничения. Даже на верхней границе входных данных время работы составляет доли секунды, а потребление памяти остаётся линейным по величине суммы. Использование бинарного разложения в ограниченной версии позволяет избежать квадратичной зависимости от количества монет и делает решение практичным на Python.

Результат работы кода:



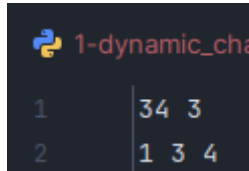
1	2 3
2	1 3 4

Рисунок 1 – Файл input.txt с входными данными из примера 1



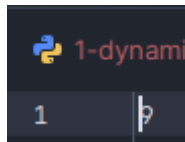
1	2
---	---

Рисунок 2 – Результат работы функции, пример из задания 1



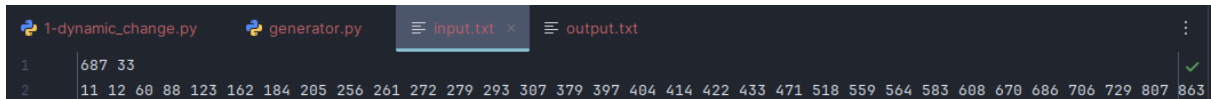
1	34 3
2	1 3 4

Рисунок 3 – Файл input.txt с входными данными из примера 2



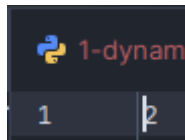
1	2
---	---

Рисунок 4 – Результат работы функции, пример из задания 2



1	687 33
2	11 12 60 88 123 162 184 205 256 261 272 279 293 307 379 397 404 414 422 433 471 518 559 564 583 608 670 686 706 729 807 863

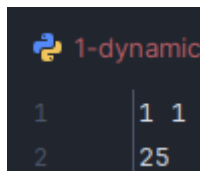
Рисунок 5 – Файл input.txt со сгенерированными входными данными (money=687, k=33)



1	2
---	---

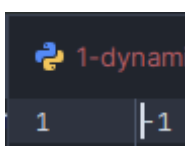
Рисунок 6 – Результат работы функции, сгенерированные входные данные

Результат работы кода на максимальных и минимальных значениях:



1	1 1
2	25

Рисунок 7 – Файл input.txt с вводными данными – минимальными значениями



1	1
---	---

Рисунок 8 – Файл output.txt после выполнения функции с минимальными значениями

Рисунок 9 – Файл input.txt с вводными данными – максимальными значениями

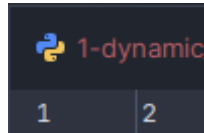


Рисунок 10 – Файл output.txt после выполнения функции с максимальными значениями

Таблица затрат памяти и времени по заданию 1:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0000 сек	0.22 KB
Сгенерированный пример	0.0059 сек	22.23 KB
Пример 1	0.0000 сек	0.29 KB
Пример 2	0.0001 сек	1.32 KB
Верхняя граница диапазона значений входных данных из текста задачи	0.0274 сек	32.22 KB

Вывод по задаче: В результате в работе реализованы и проанализированы две версии задачи обмена монет: с неограниченным и с ограниченным количеством монет. Показано, почему жадный подход не является универсальным, обоснован выбор динамического программирования и подробно рассмотрены ключевые детали реализации, обеспечивающие корректность и эффективность алгоритмов.

Задача №7. Обработка сетевых пакетов

Многие операционные системы используют шаблоны для ссылки на группы объектов: файлов, пользователей, и т. д. Ваша задача – реализовать простейший алгоритм проверки шаблонов для имен файлов.

В этой задаче алфавит состоит из маленьких букв английского алфавита и точки («.»). Шаблоны могут содержать произвольные символы алфавита, а также два специальных символа: «?» и «*». Знак вопроса («?») соответствует ровно одному произвольному символу. Звездочка «+» соответствует подстроке произвольной длины (возможно, нулевой). Символы алфавита, встречающиеся в шаблоне, отображаются на ровно один такой же символ в проверяемой строке. Строка считается подходящей под шаблон, если символы шаблона можно последовательно отобразить на символы строки таким образом, как описано выше. Например, строки «ab», «aab» и «beda.» подходят под шаблон «*a?», а строки «bebe», «a» и «ba» – нет.

- **Формат ввода / входного файла (input.txt).** Первая строка входного файла определяет шаблон. Вторая строка S состоит только из символов алфавита. Ее необходимо проверить на соответствие шаблону. Длины обеих строк не превосходят 10 000. Строки могут быть пустыми – будьте внимательны!
- **Формат вывода / выходного файла (output.txt).** Если данная строка подходит под шаблон, выведите YES. Иначе выведите NO.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Пример:

input.txt	output.txt	input.txt	output.txt
k?t*n	YES	k?t?n	NO
kitten		kitten	

Листинг кода.

```
from generator import *
from measurments import *

@measure_perf
def template_two_pointer(tmplt, strk):
    i = j = 0
    star_idx = -1
    match = 0

    n, m = len(tmplt), len(strk)

    while j < m:
        if i < n and (tmplt[i] == strk[j] or tmplt[i] == '?'):
            i += 1
            j += 1
        elif i < n and tmplt[i] == '*':
            star_idx = i
            match = j
            i += 1
        elif star_idx != -1:
            i = star_idx + 1
            match += 1
            j = match
```

```

        else:
            return 'NO'

    while i < n and tmplt[i] == '*':
        i += 1

    return 'YES' if i == n else 'NO'

@measure_perf
def template(pattern, filename):
    m = len(filename)
    n = len(pattern)
    dp = [[False for _ in range(m+1)] for _ in range(n+1)]

    dp[0][0] = True
    for j in range(1, n+1):
        if pattern[j-1] == '*':
            dp[0][j] = dp[0][j-1]

    for j in range(1, n+1):
        for i in range(1, m+1):
            if pattern[j-1] == '*':
                dp[i][j] = dp[i][j-1] or dp[i-1][j]
            elif pattern[j-1] == '?' or pattern[j-1] == filename[i-1]:
                dp[i][j] = dp[i-1][j-1]

    return "YES" if dp[n][m] else "NO"

if __name__ == '__main__':
    generate_template_and_string(t=10000, s=10**4)
    with open('input.txt') as f:
        temp = f.readline().strip()
        stroka = f.readline().strip()
    res = template_two_pointer(temp, stroka)
    with open('output.txt', 'w') as f:
        f.write(res)

```

Листинг кода дополнительной функции.

```

def generate_template_and_string(t=None, s=None):
    if t is None:
        t = random.randint(0, 10000)
    if s is None:
        s = random.randint(0, 10000)

    letters = string.ascii_lowercase + ' .'
    specials = ['?', '*']

    template = []
    for _ in range(t):
        if random.random() < 0.15:
            template.append(random.choice(specials))
        else:
            template.append(random.choice(letters))
    template = ''.join(template)

```

```
string_chars = [random.choice(letters) for _ in range(s)]
string_s = ''.join(string_chars)

with open('input.txt', 'w') as f:
    f.write(f'{template}\n{string_s}')
```

Текстовое объяснение решения.

В коде представлены два подхода к решению задачи: динамическое программирование и жадный алгоритм с двумя указателями. Функция `template`, реализующая динамическое программирование, опирается на классическую постановку задачи: вводится таблица `dp`, где `dp[i][j]` означает, возможно ли сопоставить первые i символов строки с первыми j символами шаблона. Таким образом, вся задача разбивается на подзадачи, соответствующие всем возможным парам позиций в строке и шаблоне. Переходы между состояниями отражают семантику специальных символов: для `?` и обычного символа используется переход из состояния $(i-1, j-1)$, а для `*` - объединение двух вариантов, когда `*` сопоставляется с пустой последовательностью или «поглощает» очередной символ строки. Такой подход корректен и универсален, однако требует перебора всех состояний размером $(n+1) \times (m+1)$, что приводит к квадратичной асимптотике по времени и памяти и делает его непрактичным для длинных строк.

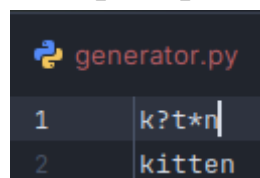
Основное решение в программе реализовано в функции `template_two_pointer`, где используется жадный алгоритм с двумя указателями. Указатель i отслеживает текущую позицию в шаблоне, а j - в строке. Дополнительно сохраняется индекс последней встреченной звёздочки `star_idx` и позиция в строке `match`, с которой эта звёздочка начала «покрывать» символы. Алгоритм последовательно сопоставляет символы строки и шаблона. Если текущие символы совпадают или в шаблоне стоит `?`, оба указателя сдвигаются вперёд. Если встречается `*`, запоминается его позиция, а алгоритм переходит к следующему символу шаблона, предполагая, что `*` пока соответствует пустой строке. В случае несовпадения, но при наличии ранее встреченного `*`, происходит контролируемый откат: шаблон возвращается к позиции сразу после `*`, а звёздочка расширяет покрываемую часть строки на один символ. Если же несовпадение возникает без возможности такого отката, сопоставление невозможно. После обработки всей строки проверяется, что оставшаяся часть шаблона состоит только из `*`, которые могут соответствовать пустой строке.

С точки зрения теории, динамическое программирование действительно разбивает задачу на подзадачи и гарантирует перебор всех

возможных состояний, но именно это делает его медленным на больших входных данных. Жадный двухуказательный алгоритм не хранит таблицу состояний и не решает все подзадачи явно, однако использует специфические свойства символа * и ограниченный откат, что позволяет фактически имитировать нужные переходы без полного перебора. За счёт этого он работает за линейное время $O(n + m)$ и практически не требует дополнительной памяти.

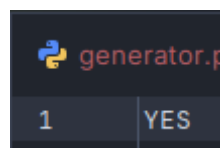
Функция генерации входных данных создаёт случайный шаблон с ограниченной вероятностью появления специальных символов и строку заданной длины, что позволяет эмпирически сравнить производительность подходов. На таких тестах динамическое программирование быстро выходит за допустимое время, тогда как двухуказательный алгоритм стабильно укладывается в ограничения.

Результат работы кода на примерах из задания:



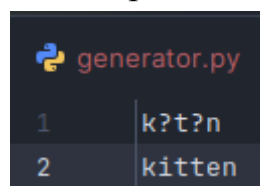
```
generator.py
1 | k?t*n
2 | kitten
```

Рисунок 11 – Файл input.txt с входными данными из примера 1



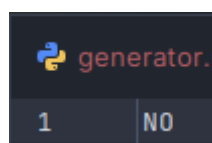
```
generator.py
1 | YES
```

Рисунок 12 – Результат работы кода для данных из примера 1, записанный в output.txt



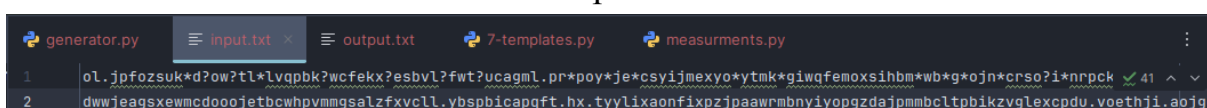
```
generator.py
1 | k?t*n
2 | kitten
```

Рисунок 13 – Файл input.txt с входными данными из примера 2



```
generator.py
1 | NO
```

Рисунок 14 – Результат работы кода для данных из примера 2, записанный в output.txt



```
generator.py | input.txt | output.txt | 7-templates.py | measurments.py
1 | ol.jpfozsuk*d?ow?tl*lvqpbk?wcfekx?esbvl?fw?ucagml.pr*poу*je*csyijmexyo*ytmk*giwgfemoxsihbm*wb*g*ojn*crso?i*nrpck
2 | dwwjjeagsxewmcdooojetbcwhpvmmsalzfxcvll.ybspbicapqft.hx.tyyliaonfixpzjpaawrbnyiyopgzdajpmbcltpbikzvglexcpdu.voethji.aojg
```

Рисунок 15 – Файл input.txt со сгенерированными входными данными (длина шаблона 7028, длина строки 3880)

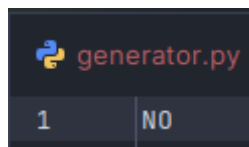


Рисунок 16 – Результат работы кода для сгенерированных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:

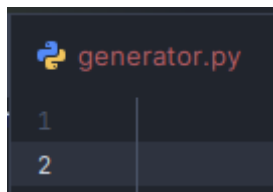


Рисунок 17 – Минимальное вводимое значение в input.txt

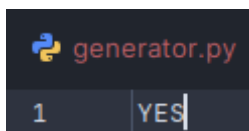


Рисунок 18 – Результат работы кода для значения s, n = 0, записанный в output.txt

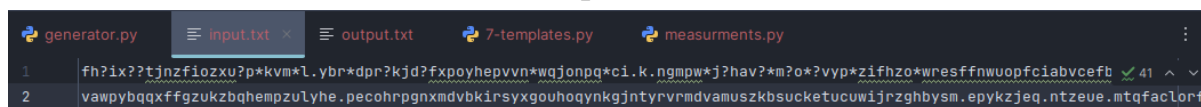


Рисунок 19 – Максимальное вводимое значение в input.txt

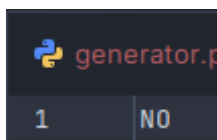


Рисунок 20 – Результат работы кода для значения s, n = 10000, записанный в output.txt

Таблица затрат памяти и времени по заданию 7:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000003 сек	0.11 KB
Пример 1	0.000006 сек	0.11 KB
Пример 2	0.000005 сек	0.11 KB

Собственный пример	0.000005 сек	0.12 KB
Верхняя граница диапазона значений входных данных из текста задачи	0.000005 сек	0.12 KB

Вывод по задаче: для задачи сопоставления строки с шаблоном при больших размерах входных данных наиболее эффективным является жадный алгоритм с двумя указателями. Динамическое программирование остаётся корректным и концептуально прозрачным решением, хорошо иллюстрирующим идею разбиения на подзадачи, но на практике уступает жадному подходу по времени и памяти.

Задачи на продвинутый уровень

Задача №3. Редакционное расстояние

Редакционное расстояние между двумя строками – это минимальное количество операций (вставки, удаления и замены символов) для преобразования одной строки в другую. Это мера сходства двух строк. У редакционного расстояния есть применения, например, в вычислительной биологии, обработке текстов на естественном языке и проверке орфографии. Ваша цель в этой задаче – вычислить расстояние редактирования между двумя строками.

- **Формат ввода / входного файла (input.txt).** Каждая из двух строк ввода содержит строку, состоящую из строчных латинских букв. Длина обеих строк - от 1 до 5000.
- **Формат вывода / выходного файла (output.txt).** Выведите расстояние редактирования между заданными двумя строками.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Примеры:

input.txt	output.txt	input.txt	output.txt	input.txt	output.txt
ab	0	short	3	editing	5
ab		ports		distance	

★ Редакционное расстояние во втором примере равна 3:

s	h	o	r	t	-
-	p	o	r	t	s

- **Дополнительное задание.** Выведите в ответ также построчно, какие действия необходимо сделать, чтобы из 1 строки получилась вторая, порядок записи действий - произвольный. Например:

input.txt	output.txt
short	3
ports	del s
	change h p
	add s

Как вариант, можно вместо действий вывести табличку как в пункте со звездочкой ★.

Листинг кода.

```
from generator import *
from measurments import *
from numba import jit

@measure_performance
@jit(nopython=True)
def lev_dist(init_word, res_word):
    n = len(init_word)
    m = len(res_word)
    if n == 0:
        return m
    if m == 0:
        return n
```

```

dp = [[0 for _ in range(m + 1)] for _ in range(n + 1)]
for i in range(n + 1):
    for j in range(m + 1):
        if i == 0 or j == 0:
            dp[i][j] = max(i, j)
        else:
            if init_word[i-1] == res_word[j-1]:
                dp[i][j] = min(dp[i - 1][j] + 1, dp[i][j - 1] + 1, dp[i - 1][j - 1])
            else:
                dp[i][j] = min(dp[i - 1][j] + 1, dp[i][j - 1] + 1, dp[i - 1][j - 1] + 1)

    return dp[n][m]

@measure_perf
@jit(nopython=True)
def ld_fast(a, b):
    if len(a) < len(b):
        a, b = b, a

    n, m = len(a), len(b)
    prev = list(range(m + 1))
    curr = [0] * (m + 1)

    for i in range(1, n + 1):
        curr[0] = i
        ai = a[i - 1]

        for j in range(1, m + 1):
            cost = 0 if ai == b[j - 1] else 1

            x = prev[j] + 1
            y = curr[j - 1] + 1
            z = prev[j - 1] + cost

            if x > y:
                x = y
            if x > z:
                x = z

            curr[j] = x

        prev, curr = curr, prev

    return prev[m]

if __name__ == '__main__':
    #levenshtein_generator(5000, 5000)
    with open('input.txt') as f:
        initial_val = f.readline().strip()
        result_val = f.readline().strip()
    res = ld_fast(initial_val, result_val)
    _ = ld_fast1(initial_val, result_val)
    with open('output.txt', 'w') as f:

```

```
f.write(str(res))
print(res)
#print(ld('short', 'ports'))
```

Листинг кода дополнительного задания

```
@measure_performance
def lev_dist_with_ops(init_word, res_word):
    n, m = len(init_word), len(res_word)

    if n == 0:
        return m, [f"add {c}" for c in res_word]
    if m == 0:
        return n, [f"del {c}" for c in init_word]

    dp = [[0] * (m + 1) for _ in range(n + 1)]
    ops = [[None] * (m + 1) for _ in range(n + 1)]

    for i in range(n + 1):
        dp[i][0] = i
        ops[i][0] = 'del'
    for j in range(m + 1):
        dp[0][j] = j
        ops[0][j] = 'add'

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if init_word[i - 1] == res_word[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
                ops[i][j] = 'match'
            else:
                candidates = [
                    (dp[i - 1][j] + 1, 'del'),
                    (dp[i][j - 1] + 1, 'add'),
                    (dp[i - 1][j - 1] + 1, 'change')
                ]
                min_val, op = min(candidates)
                dp[i][j] = min_val
                ops[i][j] = op

    operations = []
    i, j = n, m

    while i > 0 or j > 0:
        op = ops[i][j]

        if op == 'match':
            i -= 1
            j -= 1
        elif op == 'del':
            operations.append(f"del {init_word[i - 1]}")
            i -= 1
        elif op == 'add':
            operations.append(f"add {res_word[j - 1]}")
            j -= 1
        elif op == 'change':
```

```

        operations.append(f"change {init_word[i - 1]} {res_word[j - 1]}")

        i -= 1
        j -= 1

    return dp[n][m], operations

```

Листинг кода для дополнительной функции

```

def heap_generator(n=None):
    if n is None:
        n = random.randint(1, 10**6)
    arr = random.choices(range(1, 10**6), k=n)

    with open('input.txt', 'w') as f:
        f.writelines([f'{n}\n', f'{" ".join(map(str, arr))}\n'])
    return None

```

Текстовое объяснение решения.

В базовой реализации используется динамическое программирование: строится матрица $dp[i][j]$, где каждое значение - минимальное число операций для преобразования префикса первой строки длиной i в префикс второй строки длиной j . Инициализация границ матрицы выполнена отдельно ($dp[i][0] = i$, $dp[0][j] = j$), что исключает избыточные проверки внутри основного цикла. В теле цикла явно разделяются случаи совпадения символов (тогда стоимость операции нулевая) и несовпадения (тогда берётся минимум из трёх возможных операций: удаление, вставка, замена).

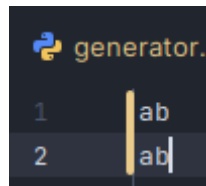
Ключевая оптимизация - переход к использованию только двух столбцов вместо полной матрицы (`ld_fast`). Это снижает потребление памяти с $O(n \cdot m)$ до $O(\min(n, m))$: хранятся лишь предыдущий (`prev`) и текущий (`curr`) столбцы. На каждой итерации по i пересчитывается текущий столбец на основе предыдущего, затем роли столбцов меняются. Это критично для больших строк: при $n = m = 5000$ память сокращается примерно с 20 МБ (полная матрица) до 40 КБ (два столбца по 5001 элементу).

Для ускорения вычислений применён декоратор `@jit` из Numba, который компилирует функцию в оптимизированный машинный код. Это даёт кратный прирост скорости (в разы) по сравнению с чистым Python, особенно на больших объёмах данных. Дополнительно минимизированы операции внутри цикла: символ `a[i - 1]` извлекается один раз перед внутренним циклом, стоимость операции (`cost`) вычисляется до ветвления, а минимум ищется через последовательные сравнения (без вызова `min()`), что сокращает накладные расходы.

Для дополнительного задания (вывод последовательности операций) построена матрица ops, хранящая тип операции (match, del, add, change) для каждой ячейки. После заполнения dp выполняется обратный проход от (n, m) к (0, 0) по ops, что позволяет восстановить точную последовательность действий. Хотя это удваивает потребление памяти, для $n, m \leq 5000$ объём остаётся в пределах лимита (около 40 МБ).

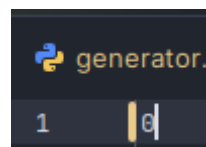
Константы и граничные случаи обработаны явно: если одна из строк пуста, расстояние равно длине другой строки. Обмен строк местами (if $\text{len}(a) < \text{len}(b)$) гарантирует, что внутренний цикл идёт по более короткой строке, минимизируя размер используемых столбцов.

Результат работы кода на примерах из задания:



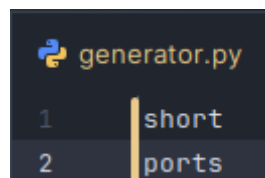
```
generator.py
1 | ab
2 | ab
```

Рисунок 21 – Файл input.txt с входными данными из примера 1



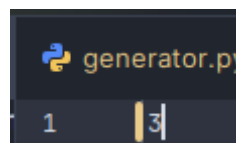
```
generator.py
1 | 3
```

Рисунок 22 – Результат работы кода для данных из примера 1, записанный в output.txt



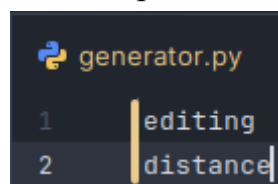
```
generator.py
1 | short
2 | ports
```

Рисунок 23 – Файл input.txt с входными данными из примера 2



```
generator.py
1 | 3
```

Рисунок 24 – Результат работы кода для данных из примера 2, записанный в output.txt



```
generator.py
1 | editing
2 | distance
```

Рисунок 25 – Файл input.txt с входными данными из примера 3

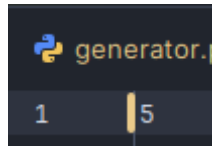


Рисунок 26 – Результат работы кода для данных из примера 3, записанный в output.txt

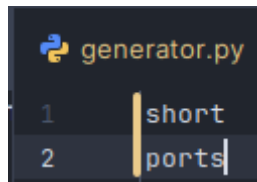


Рисунок 27 – Файл input.txt с входными данными из примера 4 (доп задание)

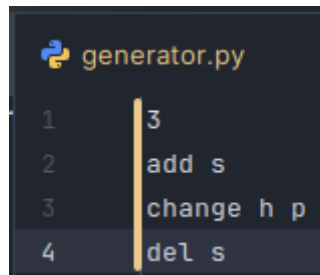


Рисунок 28 – Результат работы кода для данных из примера 4, записанный в output.txt (доп задание)

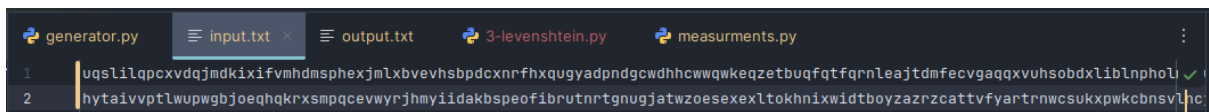


Рисунок 29 – Файл input.txt со сгенерированными входными данными (n=266, k=666)

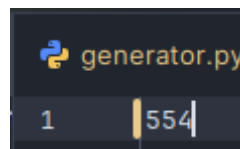


Рисунок 30 – Результат работы кода для сгенерированных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:

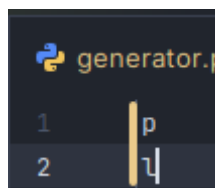


Рисунок 31 – Минимальное вводимое значение в input.txt

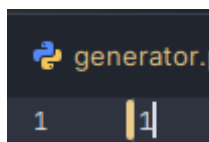


Рисунок 32 – Результат работы кода для значения $n, k = 1$, записанный в output.txt

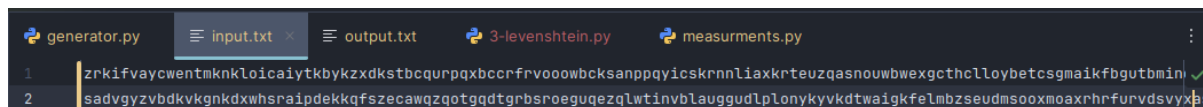


Рисунок 33 – Максимальное вводимое значение в input.txt

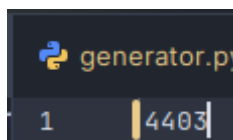


Рисунок 34 – Результат работы кода для значения $n, k = 10000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 3:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000010 сек	0.21 KB
Пример 1 из задачи	0.000013 сек	0.23 KB
Пример 2	0.000017 сек	0.27 KB
Пример 3	0.000021 сек	0.30 KB
Пример 4	0.000071 сек	0.94 KB
Сгенерированный пример	0.125155 сек	21.26 KB
Верхняя граница диапазона значений входных данных из текста задачи	1.469618 сек	0.38 MB

Вывод по задаче: задача решена с использованием классического динамического программирования, оптимизированного по памяти и по константам, а также ускоренного за счёт JIT-компиляции. Такое сочетание обеспечивает корректность решения, укладывание в ограничения по

памяти и времени и демонстрирует практическую сторону оптимизации алгоритмов при больших входных данных.

Задача №4. Наибольшая подпоследовательность двух последовательностей

Вычислить длину самой длинной общей подпоследовательности из двух последовательностей.

Даны две последовательности $A = (a_1, a_2, \dots, a_n)$ и $B = (b_1, b_2, \dots, b_m)$, найти длину их самой длинной общей подпоследовательности, т.е. наибольшее неотрицательное целое число p такое, что существуют индексы $1 \leq i_1 < i_2 < \dots < i_p \leq n$ и $1 \leq j_1 < j_2 < \dots < j_p \leq m$ такие, что $a_{i_1} = b_{j_1}, \dots, a_{i_p} = b_{j_p}$.

- **Формат ввода / входного файла (input.txt).**

- Первая строка: n - длина первой последовательности.
- Вторая строка: $a_1, a_2, \dots, a_n$ через пробел.
- Третья строка: m - длина второй последовательности.
- Четвертая строка: $b_1, b_2, \dots, b_m$ через пробел.

- Ограничения: $1 \leq n, m \leq 100$; $-10^9 < a_i, b_i < 10^9$.

- **Формат вывода / выходного файла (output.txt).** Выведите число p .

- Ограничение по времени. 1 сек.

- Примеры:

input.txt	output.txt	input.txt	output.txt	input.txt	output.txt
3	2	1	0	4	2
2 7 5		7		2 7 8 3	
2		4		4	
2 5		1 2 3 4		5 2 8 7	

- В первом примере одна общая подпоследовательность – (2, 5) длиной 2, во втором примере две последовательности не имеют одинаковых элементов. В третьем примере - длина 2, последовательности – (2, 7) или (2, 8).

Листинг кода

```
from generator import *
from measurments import *

@measure_performance
def lcs(a, b):
    n = len(a)
    m = len(b)
    table = [[0 for _ in range(m+1)] for _ in range(n+1)]
    for i in range(n+1):
        table[i][0] = 0
    for j in range(m+1):
        table[0][j] = 0

    for i in range(1, n+1):
        for j in range(1, m+1):
            if a[i - 1] == b[j - 1]:
                table[i][j] = table[i - 1][j - 1] + 1
            else:
                table[i][j] = max(table[i - 1][j], table[i][j - 1])
    return table[n][m]
```

```

if __name__ == '__main__':
    array_generator(100, 100)
    with open('input.txt') as f:
        lena = int(f.readline())
        arr_a = list(map(int, f.readline().split()))
        lenb = int(f.readline())
        arr_b = list(map(int, f.readline().split()))
    res = lcs(arr_a, arr_b)
    with open('output.txt', 'w') as f:
        f.write(str(res))
    print(res)

```

Листинг кода дополнительных функций

```

def array_generator(n=None, m=None):
    if m is None:
        m = random.randint(1, 100)
    if n is None:
        n = random.randint(1, 100)

    arra = [random.randint(-10**9 + 1, 10**9) for _ in range(n)]
    arrb = [random.randint(-10**9 + 1, 10**9) for _ in range(m)]

    with open('input.txt', 'w') as f:
        f.write(f'{n}\n{" ".join(map(str, arra))}\n{m}\n{" ".join(map(str, arrb))}')

```

Текстовое объяснение решения

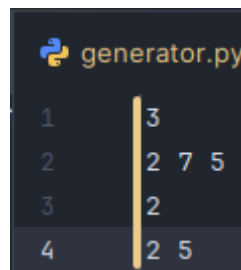
Для решения используется динамическое программирование, так как задача естественным образом раскладывается на подзадачи меньшего размера. Вводятся две последовательности a и b длины n и m соответственно. Строится двумерная таблица $table$ размером $(n+1) \times (m+1)$, где элемент $table[i][j]$ хранит длину наибольшей общей подпоследовательности для префиксов $a[0..i-1]$ и $b[0..j-1]$. Дополнительная нулевая строка и нулевой столбец используются для корректной инициализации граничных случаев, когда одна из последовательностей пуста - в этом случае длина общей подпоследовательности равна нулю.

Заполнение таблицы происходит построчно. Для каждой пары индексов (i, j) сравниваются элементы $a[i-1]$ и $b[j-1]$. Если они равны, значит найдено очередное совпадение, которое можно добавить к уже найденной общей подпоследовательности для меньших префиксов, и тогда значение в таблице увеличивается на единицу относительно $table[i-1][j-1]$. Если элементы не равны, оптимальное значение берётся как максимум из двух вариантов: либо не использовать текущий элемент из первой последовательности ($table[i-1][j]$), либо не использовать текущий элемент из второй ($table[i][j-1]$). Таким образом, на каждом шаге выбирается лучшее решение из всех допустимых подзадач.

После полного заполнения таблицы искомый ответ содержится в правом нижнем углу $table[n][m]$, так как он соответствует всей длине обеих последовательностей. Алгоритм корректно учитывает все возможные подпоследовательности, но при этом не перебирает их явно, что позволяет уложиться во временные ограничения.

В реализации используется явное построение таблицы, что при максимальных ограничениях задачи ($n, m \leq 100$) даёт сложность по времени $O(n \cdot m)$ и по памяти $O(n \cdot m)$, что полностью допустимо. Генератор входных данных создаёт последовательности случайных целых чисел в заданном диапазоне без избыточного потребления памяти, а измерение времени и памяти подтверждает, что даже на верхней границе ограничений алгоритм работает быстро и эффективно.

Результат работы кода на примерах из задания:

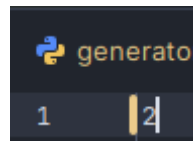


```

generator.py
1      3
2      2 7 5
3      2
4      2 5

```

Рисунок 35 – Файл input.txt с входными данными из примера 1

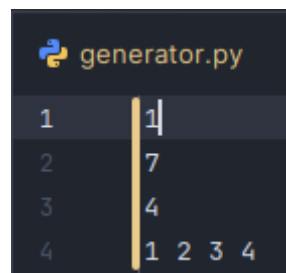


```

generator.py
1      2

```

Рисунок 36 – Результат работы кода для данных из примера 1

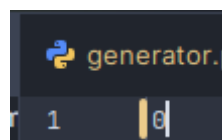


```

generator.py
1      1
2      7
3      4
4      1 2 3 4

```

Рисунок 37 – Файл input.txt с входными данными из примера 2



```

generator.py
1      6

```

Рисунок 38 – Файл output.txt с входными данными из примера 2

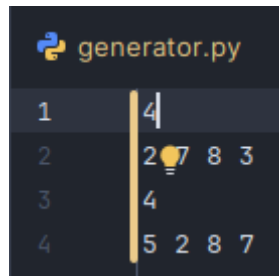


Рисунок 39 – Файл input.txt с входными данными из примера 3

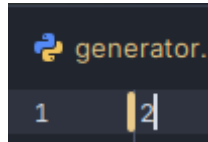


Рисунок 40 – Файл output.txt с входными данными из примера 3

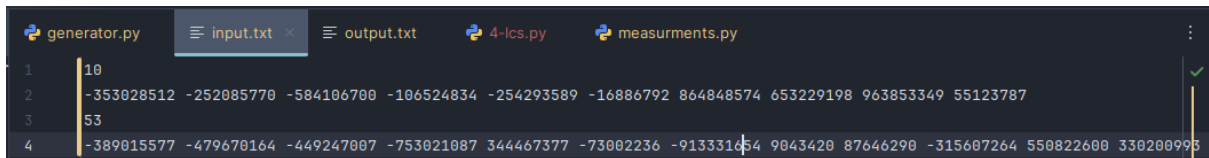


Рисунок 41 – Файл input.txt со сгенерированными входными данными (n=10, m=53)



Рисунок 42 – Файл output.txt с входными данными из примера 3 (сгенерированного)

Результат работы кода на максимальных и минимальных значениях:

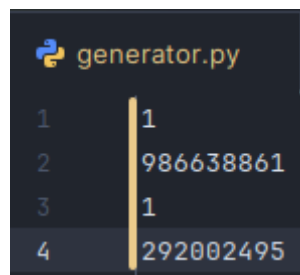


Рисунок 43 – Минимальное вводимое значение в input.txt

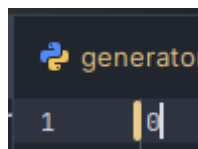


Рисунок 44 – Результат работы кода для значения n, m = 1, записанный в output.txt



Рисунок 45 – Максимальное вводимое значение в input.txt

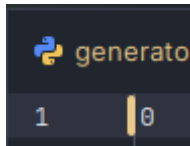


Рисунок 46 – Результат работы кода для значения $n, m = 100$, записанный в output.txt

Таблица затрат памяти и времени по заданию 4:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.000017 сек	0.28 KB
Пример 1	0.000020 сек	0.34 KB
Пример 2	0.000017 сек	0.34 KB
Пример 3	0.000030 сек	0.56 KB
Сгенерированный пример	0.000334 сек	5.81 KB
Верхняя граница диапазона значений входных данных из текста задачи	0.006048 сек	90.79 KB

Вывод по задаче: задача решена оптимальным для данных ограничений методом динамического программирования. Реализация корректна, наглядна и устойчива по времени и памяти, что подтверждается как тестовыми примерами из условия, так и автоматически сгенерированными входными данными.

Вывод

В лабораторной работе были основы динамического программирования, выявлены моменты, где жадные алгоритмы работают быстрее и эффективнее.